

INHA UNIVERSITY TASHKENT

SPRING SEMESTER 2024

SOC 2040

SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

SOC 2040

SYSTEM PROGRAMMING

WEEK 7 LECTURE 1

MACHINE LEVEL PROGRAMMING II : ADVANCED

Expressing with Goto Code

```
long absdiff
(long x, long y)
{
    long result;
    if (x > y)
        result = x-y;
    else
        result = y-x;
    return result;
}
```

- * C allows goto statement
- * Jump to position designated by label

goto
version

```
long absdiff_j
(long x, long y)
{
    long result;
    int ntest = x <= y;
    if (ntest) goto Else;
    result = x-y;
    goto Done;
Else:
    result = y-x;
Done:
    return result;
}
```

Generated assembly code

x at %rax, y at %rdx %rax ← result

```
cmpq %rdx, %rax
jge Else
subq %rax, %rdx
movq %rdx, %rax
jmp Done
```

```
Else:
    subq %rdx, %rax
```

Done:

Compare x:y
if x_ge_y goto Else
Compute result = y-x

Goto done
x_ge_y:
Compute result = x-y
Set result as return value

Done: Begin completion code

PROGRAMMING

3

General Conditional Expression Translation (Using Branches)

C Code

Using Extended C (if– goto) version

```
val = Test ? Then_Expr : Else_Expr;
```

Ternary or Conditional operator(? :)

```
val = x > y ? x - y : y - x;
```

- Create separate code regions for then & else expressions
- Execute appropriate one

Goto Version

```
n test = !Test;
if (n test) goto Else;
    val = Then_Expr;
    goto Done;
Else:
    val = Else_Expr;
Done:
    . . .
```

Goto Version : If (x <= y) goto Else;
 val = x – y;
 goto Done;
Else:
 val = y – x;
Done:

Generated assembly code

x in %rdx, y in %rbx, %rax ← result

```
cmpq %rbx, %rdx
jle Else
subq %rbx, %rdx
movq %rdx, %rax
jmp Done
```

Compare x:y
if x<=y goto Else
Compute result = x-y
Return result in %rax
Goto Done

```
Else:
    subq %rdx, %rbx
    movq %rbx, %rax
Done: .....
```

Compute result = y-x
Return result in %rax

Using Conditional Moves

★ Conditional Move Instructions

- Instruction supports:
if (Test) Dest $\leftarrow$ Src
- Supported in post-1995 x86 processors
- GCC tries to use them
 - ★ But, only when known to be safe

★ Why?

- Branches are very disruptive to instruction flow through pipelines
- Conditional moves do not require control transfer

C Code

```
val = Test ? Then_Expr : Else_Expr;
```

Goto Version

```
result = Then_Expr;  
eval = Else_Expr;  
nt = !Test;  
if (nt) result = eval;  
return result;
```

Using Conditional Moves

Conditional Move Instructions Copy the value S to its destination R when the move condition holds

Instruction	Synonym	Move condition	Description
cmove S, R	cmovz	ZF	Equal/Zero
cmovne S, R	cmovnz	~ZF	Not Equal/not Zero
cmovs S, R		SF	Negative
cmovns S, R		~SF	Nonnegative
cmovg S, R	cmovnle	~(SF^OF) & ~ZF	Greater(signed >)
cmovge S, R	cmovnl	~(SF^OF)	Greater or Equal/ (signed >=)
cmovl S, R	cmovnge	SF^OF	Less (signed <)
cmovle S, R	cmovng	(SF^OF) & ZF	Less or equal to (signed <=)
cmova S, R	cmovnbe	~CF & ~ZF	Above (Unsigned >)
cmovae S, R	cmovnb	~CF	Above or Equal (Unsigned >=)
cmovb S, R	cmovnae	CF	Below (unsigned <)
cmovbe S, R	cmovna	CF ZF	Below or equal (unsigned <=)

Conditional Move Example

```
long absdiff
(long x, long y)
{
    long result;
    if (x > y)
        result = x-y;
    else
        result = y-x;
    return result;
}
```

Conditional Move Version

```
result = x-y;
eval = y-x;
nt = !(x>y);
if (nt) result = eval;
return result;
```

Register

Use(s)

`%rdi`

Argument **x**

`%rsi`

Argument **y**

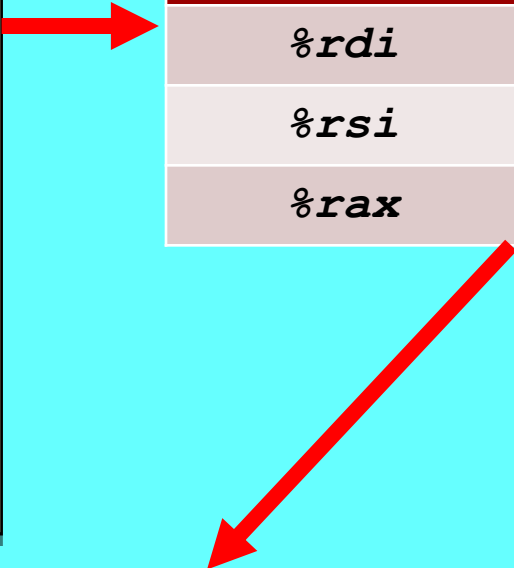
`%rax`

Return value

```
absdiff:
    movq    %rdi, %rax    # x
    subq    %rsi, %rax    # result = x-y
    movq    %rsi, %rdx
    subq    %rdi, %rdx    # eval = y-x
    cmpq    %rsi, %rdi    # x:y
    cmovle  %rdx, %rax    # if <=, result = eval
    ret
```

Conditional Move Example

```
long absdiff
(long x, long y)
{
    long result;
    if (x > y)
        result = x-y;
    else
        result = y-x;
    return result;
}
```



Register	Use(s)
<code>%rdi</code>	Argument <code>x</code>
<code>%rsi</code>	Argument <code>y</code>
<code>%rax</code>	Return value

absdiff:

Core0

```
movq    %rdi, %rax
subq    %rsi, %rax
cmpq    %rsi, %rdi
cmovle  %rdx, %rax
ret
```

Core1

```
movq    %rsi, %rdx
subq    %rdi, %rdx
```

Parallel execution on two cores

MACHINE-LEVEL PROGRAMMING II : LOOP STRUCTURES

General “Do-While” Translation

C Code

```
do  
    Body  
while ( Test );
```

Goto Version

```
loop:  
    Body  
    if ( Test )  
        goto loop
```

* Body:

```
{  
    Statement1;  
    Statement2;  
    ...  
    Statementn;  
}
```

“Do-While” Loop Example

- * Count number of 1's in argument *x* (“popcount”)
- * Use conditional branch to either continue looping or to exit loop

C Code

```
long pcount_do
(unsigned long x) {
    long result = 0;
    do {
        result += x & 0x1;
        x >>= 1;
    } while (x);
    return result;
}
```

Goto Version

```
long pcount_goto
(unsigned long x) {
    long result = 0;
    loop:
        result += x & 0x1;
        x >>= 1;
        if(x) goto loop;
    return result;
}
```

“Do-While” Loop Example

* Count number of 1's in argument **x**

Register Mapping :

x in **%rdi** and result in **%rax**

```
long pcount_goto
(unsigned long x) {
    long result = 0;
loop:
    result += x & 0x1;
    x >>= 1;
    if(x) goto loop;
    return result;
}
```

Given **x = 00001011**

%rdi = x = 00001011

%rax=0 = result

Loop: **%rdx = x & 0x1 = %rdi & 0x1**
 = 00001011 & 00000001
 = 00000001 = 1

%rax = %rax + %rdx = 0 + 1 = 1

shrq %rdi → %rdi= 00000101

if %rdi != 0 goto Loop

Loop: **%rdx = x & 0x1 = %rdi & 0x1**
 = 00000101 & 00000001
 = 00000001 = 1

%rax = %rax + %rdx = 1 + 1 = 2

shrq %rdi → %rdi= 00000010

if %rdi != 0 goto Loop

Loop: **%rdx = x & 0x1 = %rdi & 0x1**
 = 00000010 & 00000001
 = 00000000 = 0

%rax = %rax + %rdx = 2 + 0 = 2

shrq %rdi → %rdi= 00000001

if %rdi != 0 goto Loop

Loop: **%rdx = x & 0x1 = %rdi & 0x1**
 = 00000001 & 00000001
 = 00000001 = 1

%rax = %rax + %rdx = 2 + 1 = 3

shrq %rdi → %rdi= 00000000

if %rdi = 0 return result = %rax = 3

“Do-While” Loop Compilation

Goto Version

```
long pcount_goto
(unsigned long x) {
    long result = 0;
loop:
    result += x & 0x1;
    x >>= 1;
    if(x) goto loop;
    return result;
}
```

Register	Use(s)
<i>%rdi</i>	Argument <i>x</i>
<i>%rax</i>	<i>result</i>

```
    movq    $0, %rax           # result = 0
Loop:                                     # loop
    movq    %rdi, %rdx
    andq    $1, %rdx           # t = x & 0x1
    addq    %rdx, %rax         # result += t
    shrq    %rdi               # x >>= 1
    jne     Loop              # if (x) goto Loop
    ret
```

“Do-While” Loop Compilation

Goto Version

a) C code

```
int fact_do(int n)
{
    int result = 1;
    do {
        result *= n;
        n = n-1;
    } while (n > 1);
    return result;
}
```

Register	Variable	Initially
<code>%rax</code>	<code>result</code>	<code>1</code>
<code>%rdx</code>	<code>n</code>	<code>n</code>

(c) Corresponding assembly-language code

Registers: `n` in `%rdx`, `result` in `%rax`

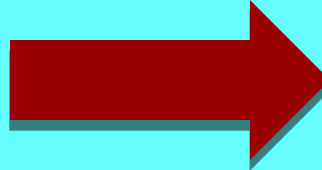
<code>movq \$1, %rax</code>	Set <code>result = 1</code>
Loop: <code>imulq %rdx, %rax</code>	loop: Compute <code>result *= n</code>
<code>subq \$1, %rdx</code>	Decrement <code>n</code>
<code>cmpq \$1, %rdx</code>	Compare <code>n:1</code>
<code>jg Loop</code>	If <code>></code> , goto loop
<code>ret</code>	Return result

General “While” Translation #1

- ★ “Jump-to-middle” translation

While version

```
while (Test)  
    Body
```



Goto Version

```
    goto test;  
loop:  
    Body  
test:  
    if (Test)  
        goto loop;  
done:
```

While Loop Example #1

- * Compare to do-while version of function - Initial goto starts loop at test

C Code

```
long pcount_while
(unsigned long x) {
    long result = 0;
    while (x) {
        result += x & 0x1;
        x >>= 1;
    }
    return result;
}
```

Jump to Middle

```
long pcount_goto_jtm
(unsigned long x) {
    long result = 0;
    goto test;
loop:
    result += x & 0x1;
    x >>= 1;
test:
    if(x) goto loop;
    return result; }
```

(c) Corresponding assembly-language code

Registers: x in %rdi, result in %rax

```
    movq    $0, %rax           # %rax = result = 0
    movq    x, %rdi            # %rdi = x
    jmp     Test               # goto Test
Loop: movq    %rdi, %rdx        # %rdx = %rdi
    andq    $1, %rdx           # %rdx = %rdx & 0x1
    addq    %rdx, %rax          # %rax += %rdx
    shrq    %rdi               # %rdi >>= 1
Test: jne     Loop             # if (%rdi > 0) goto Loop
    ret
```


"While" Loop Compilation

a) C code

```
long fact_while(long n)
{
    long result = 1;
    while (n > 1)
    {
        result *= n;
        n = n-1;
    }
    return result;
}
```

b) Equivalent goto version

```
long fact_while_goto(long n)
{
    long result = 1;
    goto test;

loop:
    result *= n;
    n = n-1;

test:
    if (n > 1) goto loop;
    return result;
}
```

(c) Corresponding assembly-language code - long fact_while(long n)

Argument: n in %rdi

fact_while:

```
    movq $1, %rax
    jmp Test
```

Set result = 1
goto test

Loop:

```
    imulq %rdi, %rax
    subq $1, %rdi
```

Compute result *= n
Decrement n

Test:

```
    cmpq $1, %rdi
    jg Loop
    ret
```

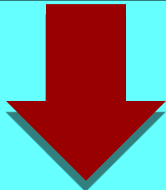
Compare n:1
If >, goto loop
Return result

Register	Variable	Initially
%rax	result	1
%rdi	n	n

General “While” Translation #2

While version

```
while (Test)  
    Body
```



Do-While Version

```
if (!Test)  
    goto done;  
do  
    Body  
    while (Test) ;  
done:
```



Goto Version

```
if (!Test)  
    goto done;  
loop:  
    Body  
    if (Test)  
        goto loop;  
done:
```

★ “Do-while” conversion

While Loop Example #2

* Compare to do-while version of function- Initial conditional guards entrance to loop

C Code

```
long pcount_while
(unsigned long x) {
    long result = 0;
    while (x) {
        result += x & 0x1;
        x >>= 1;
    }
    return result;
}
```

Do-While if-goto Version

```
long pcount_goto_dw
(unsigned long x) {
    long result = 0;
    if (!x) goto done;
loop:
    result += x & 0x1;
    x >>= 1;
    if(x) goto loop;
done:
    return result;
}
```

(c) Corresponding assembly-language code

Registers: x in %rdi, result in %rax

	movq	\$0, %rax	# %rax = result = 0
	cmpq	\$0, %rdi	# compare %rdi with 0
	je	Done	# goto Done if %rdi=x=0
Loop:	movq	%rdi, %rdx	# %rdx = %rdi
	andq	\$1, %rdx	# %rdx = %rdx & 0x1
	addq	%rdx, %rax	# %rax += %rdx
	shrq	%rdi	# %rdi >>= 1
	jne	Loop	# if (%rdi > 0) goto Loop
Done:	ret		

“While” Loop Compilation

a) C code

```
int fact_while(int n)
{
    int result = 1;
    while (n > 1)
    {
        result *= n;
        n = n-1;
    }
    return result;
}
```

b) Equivalent goto version

```
int fact_while_goto(int n)
{
    int result = 1;
    if (n <= 1) goto done;

loop:
    result *= n;
    n = n-1;
    if (n > 1)
        goto loop;

done: return result;
}
```

(c) Corresponding assembly-language code

Registers: n in %rdx, result in %rax 1

```
movq $1, %rax
cmpq $1, %rdx
jle Done
```

Loop:

```
imulq %rdx, %rax
subq $1, %rdx
cmpq $1, %rdx
jg Loop
```

Done:

```
Set result = 1
Compare n:1
If <=, goto done
```

loop:

```
Compute result *= n
Decrement n
Compare n:1
If >, goto loop
```

Return result

Register	Variable	Initially
%rax	result	1
%rdx	n	n

"For" Loop Form

General Form

```
for (Init; Test; Update )  
    Body
```

```
#define WSIZE 8*sizeof(long int)  
long pcount_for  
    (unsigned long x)  
{  
    size_t i;  
    long result = 0;  
    for (i = 0; i < WSIZE; i++)  
    {  
        unsigned bit =  
            (x >> i) & 0x1;  
        result += bit;  
    }  
    return result;  
}
```

Init

```
i = 0
```

Test

```
i < WSIZE
```

Update

```
i++
```

Body

```
{  
    unsigned bit =  
        (x >> i) & 0x1;  
    result += bit;  
}
```

“For” Loop → While Loop

For Version

```
for (Init; Test; Update )  
    Body
```



While Version

```
Init;  
while (Test) {  
    Body  
    Update;  
}
```

For-While Conversion

Init

```
i = 0
```

Test

```
i < WSIZE
```

Update

```
i++
```

Body

```
{  
    unsigned bit =  
        (x >> i) & 0x1;  
    result += bit;  
}
```

```
long pcount_for_while  
(unsigned long x)  
{  
    size_t i;  
    long result = 0;  
    i = 0;  
    while (i < WSIZE)  
    {  
        unsigned bit =  
            (x >> i) & 0x1;  
        result += bit;  
        i++;  
    }  
    return result;  
}
```

"For" Loop Compilation

a) C code for loop

```
long fact_for(long n)
{
    long i,
    long result = 1;
    for(i=2; i<=n; i++)
        result *= i;
    return result;
}
```

b) for loop into while loop

```
long fact_for_while(long n)
{
    long i=2, result = 1;
    while (i <= n){
        result *= i;
        i++; }
    return result;
}
```

c) for loop -Goto version

```
long fact_for_goto(long n)
{
    long i = 2;
    long result = 1;
    goto test;

loop:
    result *= i;
    i++;

test:
    if(i <= n)
        goto loop;
    return result;
}
```

(d) Assembly code

long fact_for(long n)

Argument: n in %rdi

fact_for:

movq \$1, %rax	Set result = 1
movq \$2,%rdx	set i=2
jmp .L8	goto test
.L9:	
imulq %rdx, %rax	Compute result *= i
addq \$1, %rdx	increment i
.L8:	
cmpq %rdi,%rdx	Compare i:n
jle .L9	If <=, goto loop
ret	Return result

“For” Loop Do-While Conversion

Goto Version

C Code

```
long pcount_for
(unsigned long x)
{
    size_t i;
    long result = 0;
    for (i = 0; i < WSIZE; i++)
    {
        unsigned bit =
            (x >> i) & 0x1;
        result += bit;
    }
    return result;
}
```

```
long pcount_for_goto_dw
(unsigned long x) {
    size_t i;
    long result = 0;
    i = 0; Init
```

```
    loop:
    {
        unsigned bit =
            (x >> i) & 0x1;
        result += bit; Body
    }
    i++; Update
    if (i < WSIZE)
        goto loop; Test
done:
    return result;
}
```

- * Initial test can be optimized away

MACHINE-LEVEL PROGRAMMING II : SWITCH STRUCTURE

Switch Statement Example

- ★ **A switch statement provides a multi-way branching capability based on the value of an integer index.**
- ★ They are particularly useful when dealing with tests where there can be a large number of possible outcomes.
- ★ Not only do they make the C code more readable, they also allow an efficient implementation using a data structure called a jump table.
- ★ A jump table is an array where entry *i* is the address of a code segment implementing the action the program should take when the switch index equals *i*.
- ★ The code performs an array reference into the jump table using the switch index to determine the target for a jump instruction.
- ★ The advantage of using a jump table over a long sequence of if-else statements is that the time taken to perform the switch is independent of the number of switch cases.

Jump Table Structure

Switch Form

```
switch(x) {  
  case val_0:  
    Block 0  
  case val_1:  
    Block 1  
    . . .  
  case val_n-1:  
    Block n-1  
}
```

Jump Table

jtab:	Targ0
	Targ1
	Targ2
	•
	•
	•
	Targn-1

Jump Targets

Targ0: Code Block 0

Targ1: Code Block 1

Targ2: Code Block 2

•
•
•

Targn-1: Code Block n-1

Translation (Extended C)

```
goto *jtab[x];
```

Switch Statement Example

Translation into Extended C

```
long switch_eg
(long x, long n, long *dest)
{
    long val = x;
    switch(n) {
        case 100:
            val *= 13;
            break;
        case 102:
            val += 10;
            /* Fall Through */
        case 103:
            val += 11;
            break;
        case 104:
        case 106:
            val *= val;
            break;
        default:
            val = 0;
    }
    *dest = val;
}
```

```
void switch_eg_impl
(long x, long n, long *dest)
{
    /*Table of code pointers*/
    Static void *jt[7] = {
        &&Loc_A, &&Loc_def, &&Loc_B, &&Loc_C,
        &&Loc_D, &&Loc_def, &&Loc_D };

    Unsigned long index = n-100;
    long val;
    if (index > 6)
        goto Loc_def;
    /*Multiway branch */
    goto *jt[index];
    Loc_A: /*Case 100    index=0 */
        val = x + 13;
        goto done;
    Loc_B: /*Case 102    index=2 */
        x = x + 10;
        /* Fall Through */
    Loc_C: /*Case 103    index=3 */
        val = x + 11;
        goto done;
    Loc_D: /*Cases 104,106 index=4,6 */
        val = x * x;
        goto done;
    Loc_def: /*Default Case */
        val = 0;

    done :
        *dest = val; }
```

Switch Statement Example

Assembly Code for switch statement

Jump Table

```
void switch_eg(long x, long n, long *dest)
x in %rdi,  n in %rsi,  dest in %rdx
```

Switch_eg:

```
    subq $100, %rsi          Compute index = n-100
    cmpq $6, %rsi            Compare index:6
    ja   L8                  If >, goto loc_def
    jmp  *L4(,%rsi,8)         Goto *jg[index]
```

```
L3:    leaq (%rdi,%rdi,2),%rax  3*x
       leaq (%rdi,%rax,4),%rdi  val=13*x
       jmp  L2                 Goto done
```

```
L5:    addq $10, %rdi          x = x + 10
```

```
L6:    addq $11, %rdi          val = x + 11
       jmp  L2                 Goto done
```

```
L7:    imulq %rdi, %rdi        val = x * x
       jmp  L2                 Goto done
```

```
L8:    movl $0, %edi           Loc_def:
                                     val = 0
```

```
L2:    movq %rdi, (%rdx)       done:
    ret                        *dest = val
```

```
.section .rodata
.align 8  Align address to multiple of 8
L4:
.quad    L3      #case 100: Loc_A
.quad    L8      #case 101: Loc_def
.quad    L5      #case 102: Loc_B
.quad    L6      #case 103: Loc_C
.quad    L7      #case 104: Loc_D
.quad    L8      #case 105: Loc_def
.quad    L7      #case 106: Loc_D
```

Register	Use(s)
<i>%rdi</i>	Argument <i>x</i>
<i>%rsi</i>	Argument <i>n</i>
<i>%rdx</i>	dest
dest - memory location to store result in %rdi	

Assembly Setup Explanation

* Table Structure

- Each target requires 8 bytes
- Base address at **L4**

* Jumping

- Direct:** `jmp L8`
- Jump target is denoted by label **.L8**
- Indirect:** `jmp *L4(, %rdi, 8)`
- Start of jump table: **L4**
- Must scale by factor of 8 (addresses are 8 bytes)
- Fetch target from effective Address **L4 + x*8**
 - *Only for $0 \leq x \leq 6$

Jump table

```
.section .rodata
    .align 8
L4:
    .quad    L3
    # x = 0
    .quad    L8
    # x = 1
    .quad    L5
    # x = 2
    .quad    L6
    # x = 3
    .quad    L7
    # x = 4
    .quad    L8
    # x = 5
    .quad    L7
    # x = 6
```

Summarizing

- ★ C Control
 - if-then-else
 - do-while
 - while, for
 - switch
- ★ Assembler Control
 - Conditional jump
 - Conditional move
 - Indirect jump (via jump tables)
 - Compiler generates code sequence to implement more complex control
- ★ Standard Techniques
 - Loops converted to do-while or jump-to-middle form
 - Large switch statements use jump tables
 - Sparse switch statements may use decision trees (if-elseif-elseif-else)

SOC 2040 SYSTEM PROGRAMMING

Reading Assignments

Chapter 3 of Text Book

➤ Computer Systems : A Programmer's Perspective

- Randal E. Bryant and David R. O'Hallaron
3rd Edition, Global Edition Prentice Hall 2016
ISBN 10:1-292-10176-8**

INHA UNIVERSITY TASHKENT

SPRING SEMESTER 2024

SOC 2040

SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

